

Hybrid Secure Voting System

Ami jinista nia kaj korte chai..

Somossa and samadhan er upay gula sundor kore likho..

Pore amake bolo ki bhabo kaj suru korte pari

--+++

Ballote paper instance make hobe.muhurite nid verification / voter nong verification thakbe. Ballote er main page e anno momus number dwara verified hobe / mhuri machine e thakbe only verified ballote print hobe.

-fingerprint er maddhome+ retina scan + face verifications

-Ballote machine e jomadike tokhon e ballote qr er sathe scan kore milabe + thokhon e vote machine counting hoye jabe.

Counting app:

Oi kendre koyta ballote paper ber hoise tar hisab thakbe.

Counting machine er ballote count officer er kache thakbe + app o show korbe.

Vote dear time e kon marka koto vote paise ta show korbe na but kotojon vote dise ta show korbe.

Manual gonona hobe. + jotogula agent thakbe tader sobai sob doler propto vote input dibe.

তুমি যে সিস্টেমটা ভাবছো, এটা মূলত একটি **Hybrid Secure Voting System** — যেখানে

- Pre-verified ballot printing
- Biometric authentication
- QR-based tracking
- Real-time turnout monitoring
- Manual counting + Digital cross-verification

আমি বিষয়টা ৩ অংশে পরিষ্কার করে দিচ্ছি:

☐☐ সম্ভাব্য সমস্যা (Technical + Operational Issues)

1. Biometric Verification Risk

সমস্যা:

- Fingerprint mismatch (ঘাম, কাটা দাগ, বৃদ্ধ মানুষ)
- Retina scanner expensive + power dependent
- Face recognition lighting sensitive

- False reject / false accept rate

ঝুঁকি:

- ভোটের দাঁড়িয়ে থাকবে কিন্তু verify হবে না
 - ভোট দিতে না পারলে সামাজিক অস্থিরতা
-

2. QR + Ballot Syncing Problem

সমস্যা:

- QR clone করা সম্ভব (যদি encryption না থাকে)
 - Scanner delay হলে লাইনে জট
 - Paper tear হলে QR unreadable
-

3. Real-Time Counting Visibility Risk

সমস্যা:

- যদি turnout data বাইরে leak হয় → প্রভাব ফেলতে পারে
 - Network hacking risk
 - Server crash হলে সমস্যা
-

4. Hardware Dependency

সমস্যা:

- বিদ্যুৎ না থাকলে?
 - Machine damage হলে?
 - গ্রামীণ এলাকায় connectivity কম
-

5. Manual + Digital Data Mismatch

সমস্যা:

- Manual count $\neq$ App data হলে বিতর্ক
 - Multiple agents input দিলে conflict
-

☐ সম্ভাব্য সমাধান (Structured Solutions)

✓ 1. Multi-Layer Authentication Design

- 1st Layer → NID / Voter ID number
 - 2nd Layer → Fingerprint (Primary)
 - Backup → Face recognition only (Retina optional, not mandatory)
 - Offline biometric database (local cache)
-

✓ 2. Secure Ballot Printing System

- Unique encrypted QR (AES encrypted)
 - QR contains:
 - Center ID
 - Serial number
 - Timestamp
 - Random hash
 - Only verified voter → machine prints ballot
 - Pre-printed ballot নয়
-

✓ 3. Smart Counting Logic

Voting time:

- Only show:
 - Total voters cast vote
 - Total ballot printed
- NOT show party-wise vote

After poll close:

- Manual counting
- Agents input vote count
- System cross-check:
 - Printed ballot সংখ্যা
 - Box deposited ballot সংখ্যা
 - Agent entered vote সংখ্যা

Mismatch হলে → auto flag

✓ 4. Power & Connectivity Backup

- UPS (4–6 hour backup)
 - Offline mode + later sync
 - Data stored in encrypted local storage
-

✓ 5. Transparency Mechanism

- Counting machine data visible to Presiding Officer
 - Counting App visible to central authority
 - All agents must digitally sign result
-

📋 পুরো সিস্টেমটা কেমন হবে (Workflow Overview)

Step 1 → Voter arrives

Step 2 → NID verification

Step 3 → Fingerprint verify

Step 4 → Ballot printed with encrypted QR

Step 5 → Voter votes manually

Step 6 → Ballot inserted into scanning ballot box

Step 7 → QR scanned → turnout count increment

Step 8 → After poll → Manual counting + Digital reconciliation

📋 এখন প্রশ্ন — তুমি কিভাবে কাজ শুরু করবে?

তুমি যেহেতু coding background এর (problem solving করো), তাই আমি structured roadmap দিচ্ছি।

Phase 1: Concept Design (2–3 weeks)

- ✓ System architecture diagram আঁকো
- ✓ Database schema design করো
- ✓ QR data structure define করো
- ✓ Security threat model লিখো

Tools:

- Draw.io

- Notion
 - ER diagram
-

Phase 2: Prototype Development

Module 1 → Voter Verification App

- Android app
- Local SQLite DB
- Fingerprint SDK (simulation)

Module 2 → Ballot Print Simulation

- QR generate (Java/Python)
- Unique hash generate

Module 3 → Counting App

- Show only turnout
 - After poll → enable vote input
-

Phase 3: Security Layer

- Encryption implementation
 - Role-based login
 - Audit log system
-

Phase 4: Pilot Simulation

- 50 dummy voters
 - Test:
 - Biometric failure
 - QR mismatch
 - Double voting attempt
 - Power failure scenario
-

গুরুত্বপূর্ণ রেফারেন্স সিস্টেম



Election Commission of India

- EVM + VVPAT model ব্যবহার করে



Tribunal Superior Eleitoral

- Fully electronic voting



Estonian National Electoral Committee

- Internet voting model



গুরুত্বপূর্ণ সতর্কতা

এটা শুধুমাত্র প্রযুক্তিগত প্রজেক্ট হিসেবে ভাবো।

Real national election system বানাতে গেলে লাগে:

- আইনগত অনুমোদন
- জাতীয় পর্যায়ের infrastructure
- High-grade security audit
- International certification



আমার পরামর্শ (তোমার জন্য)

তুমি এখন যা করতে পারো:

1. এটাকে final year project বানাও
2. শুধু “Smart Hybrid Ballot System Prototype” বানাও
3. Complete system না, modular demo বানাও
4. GitHub repo খুলে documentation শুরু করো

চাও কি আমি তোমার জন্য:

- Full system architecture diagram লিখে দেই?
- Database schema design করি?
- QR encryption logic example দেই?
- Android prototype flow লিখে দেই?

বললে আমরা এখনই next step শুরু করতে পারি.

তুমি যেহেতু পুরো Hybrid Secure Ballot System করতে চাও, আমি এখানে **Production-Level Architecture** দিচ্ছি — layered, modular, scalable ডিজাইন ।

আমি এটাকে ৫টা Layer + ৯টা Core Module এ ভেঙে দিচ্ছি ।

FULL SYSTEM ARCHITECTURE (Hybrid Secure Ballot System)

1 High Level Architecture (Layer View)

pgsql

Presentation Layer
- Voter Verification App
- Ballot Printing Interface
- Counting Officer App
- Agent Input App

Application Layer
- Authentication Engine
- QR Generator & Validator
- Turnout Counter Service
- Vote Reconciliation Engine
- Audit Log Manager

Security Layer
- Biometric Verification Engine
- Encryption Service (AES- 256)
- Digital Signature Service
- Role -Based Access Control (RBAC)

Data Layer	
- Local Encrypted DB (Polling Center)	
- Central Election Server DB	
- Offline Sync Queue	

Infrastructure Layer	
- Ballot Printer Machine	
- Biometric Device	
- QR Scanner Ballot Box	
- UPS Backup	
- Secure Network Gateway	

2 Component-Level Architecture (Detailed Module View)

Module 1: Voter Verification System

Input:

- NID / Voter ID
- Fingerprint
- Face scan (backup)

Process:

1. Fetch voter record
2. Biometric match
3. Check “Already Voted” flag
4. If valid → Generate Voting Token

Output:

- Temporary encrypted Voting Token (valid for 5 mins)

Module 2: Secure Ballot Printing System

Flow:

Voting Token → Validate → Generate Ballot ID →
Encrypt → Generate QR → Print Ballot

QR Contains:

- Center_ID
- Ballot_Serial
- Timestamp
- Random Hash
- Digital Signature

Encrypted using:

AES-256 + HMAC

Module 3: Smart Ballot Box + QR Scanner

যখন ভোটার ballot ঢুকাবে:

1. QR Scan
2. Decrypt
3. Validate:
 - Same center?
 - Not used before?
4. If valid:
 - Increment turnout counter
 - Mark Ballot as Cast
5. If invalid:
 - Reject alert

Module 4: Turnout Monitoring System

Real-time show করবে:

- Total ballot printed
- Total ballot cast
- Remaining voters

 Party-wise vote show করবে না

Module 5: Manual Counting Reconciliation Engine

Poll close হলে:

1. Manual count by officers
2. Agents input party-wise votes
3. System cross-check:

mathematica

Total Cast Ballot ==
Sum(Party Votes)

4. Mismatch → Red Flag

Module 6: Role-Based Access Control (RBAC)

Roles:

Role	Permission
Presiding Officer	Full center control
Counting Officer	View turnout + count
Party Agent	Input party vote
Admin (Central)	System oversight

Database Architecture

Polling Center Local DB

Tables:

Voter_Table

- voter_id
- name
- fingerprint_hash
- has_voted (boolean)

Ballot_Table

- ballot_id
- center_id
- timestamp
- status (Printed / Cast / Invalid)

Audit_Log

- event_id
- event_type
- user_role
- timestamp



Central Election DB

- Center Summary Table
- Turnout Log
- Final Result Table
- Discrepancy Report Table



4 Security Architecture

mathematica

Biometric → Token → Ballot → QR → Scan → Validation

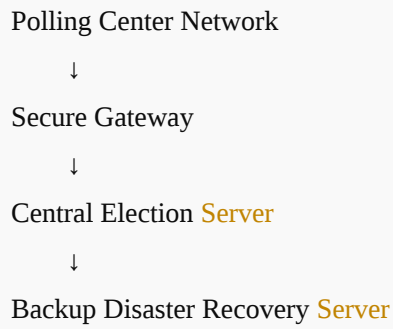
Security Layers:

1. Device-level encryption
2. Secure boot firmware
3. Offline signed firmware
4. Data encryption at rest
5. Data encryption in transit (TLS 1.3)
6. Digital signature verification



5 Network Architecture

arduino



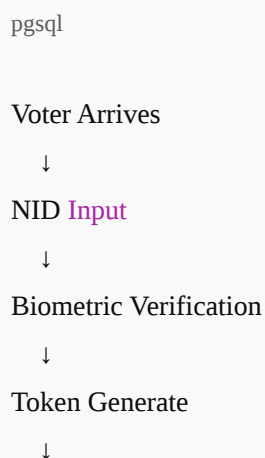
Offline mode:

- All data stored locally
- After network restore → Sync

6 Failure Handling Architecture

Failure	Handling
Power failure	UPS + Local Save
Biometric failure	Secondary verification
QR unreadable	Manual ballot verification
Server down	Offline mode
Duplicate ballot	System auto reject

7 Complete Workflow Diagram (Process Flow)



Ballot Print (Encrypted QR)



Vote Cast (Manual Mark)



Ballot Box Scan



Turnout Increment



Poll Close



Manual Counting



Agent Input



System Cross Verification



Final Result Lock

8 Scalability Design

- Each polling center → independent node
- Central server → only aggregate
- No live vote reveal
- Horizontal scaling supported

9 Development Technology Stack Suggestion

Layer	Tech
Frontend	Android (Java/Kotlin)
Backend	Spring Boot / Node.js
DB	PostgreSQL
Encryption	Java Crypto API
QR	ZXing Library

Layer	Tech
Biometric	Device SDK

তোমার জন্য বাস্তবিক পরামর্শ

তুমি একবারে national-level system বানানোর চেষ্টা করো না।

Start with:

- ✓ 1 Polling Center Simulation
- ✓ Fake voter DB
- ✓ QR encrypted ballot
- ✓ Turnout counting
- ✓ Manual reconciliation

যদি চাও, next আমি দিতে পারি:

- ER Diagram
- Database Schema (SQL ready)
- Microservice architecture version
- Threat model document
- GitHub project structure
- Android prototype structure

কোনটা আগে দরকার?

database schema design kore daw.

আমি এখানে তোমার Hybrid Secure Ballot System এর জন্য **Production-Grade Relational Database Schema** দিচ্ছি।

ডিজাইন Goal:

- Data Integrity
- No double voting
- Audit traceability

- Manual + Digital reconciliation support
- Offline center DB + Central DB separation

আমি ২ ভাগে দিচ্ছি:

1 Polling Center Local Database

2 Central Election Database

PART 1: Polling Center Local Database Schema

1 voters

sql

```
CREATE TABLE voters (  
  voter_id VARCHAR(20) PRIMARY KEY,  
  nid_number VARCHAR(20) UNIQUE NOT NULL,  
  full_name VARCHAR(150) NOT NULL,  
  fingerprint_hash TEXT NOT NULL,  
  face_hash TEXT,  
  center_id VARCHAR(20) NOT NULL,  
  has_voted BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Purpose:

- Local cached voter list
- Prevent double voting (has_voted flag)

2 voting_tokens

sql

```
CREATE TABLE voting_tokens (  
  token_id UUID PRIMARY KEY,  
  voter_id VARCHAR(20) REFERENCES voters(voter_id),  
  issued_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  expires_at TIMESTAMP NOT NULL,
```

```
used BOOLEAN DEFAULT FALSE  
);
```

Purpose:

- Short-lived authorization
 - Prevent replay attack
-

3 ballots

sql

```
CREATE TABLE ballots (  
  ballot_id UUID PRIMARY KEY,  
  center_id VARCHAR(20) NOT NULL,  
  voter_id VARCHAR(20) REFERENCES voters(voter_id),  
  serial_number BIGSERIAL UNIQUE,  
  qr_hash TEXT NOT NULL,  
  status VARCHAR(20) CHECK (status IN ('PRINTED','CAST','INVALID')) DEFAULT 'PRINTED',  
  printed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  cast_at TIMESTAMP  
);
```

Purpose:

- Track each ballot lifecycle
 - Ensure 1 voter = 1 ballot
-

4 turnout_log

sql

```
CREATE TABLE turnout_log (  
  id SERIAL PRIMARY KEY,  
  total_printed INTEGER DEFAULT 0,  
  total_cast INTEGER DEFAULT 0,  
  last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Purpose:

- Quick dashboard query
 - No party vote stored here
-

5 manual_count_entries

sql

```
CREATE TABLE manual_count_entries (  
  entry_id SERIAL PRIMARY KEY,  
  party_id VARCHAR(20),  
  vote_count INTEGER NOT NULL,  
  entered_by VARCHAR(50),  
  entry_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Purpose:

- Agents input party-wise votes
 - Multiple entries allowed
-

6 parties

sql

```
CREATE TABLE parties (  
  party_id VARCHAR(20) PRIMARY KEY,  
  party_name VARCHAR(100) NOT NULL,  
  symbol VARCHAR(100)  
);
```

7 users (RBAC)

sql

```
CREATE TABLE users (  
  user_id VARCHAR(50) PRIMARY KEY,  
  password_hash TEXT NOT NULL,  
  role VARCHAR(30) CHECK (role IN
```

```
('PRESIDING_OFFICER','COUNTING_OFFICER','AGENT','ADMIN')),  
  center_id VARCHAR(20),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

8 audit_logs

sql

```
CREATE TABLE audit_logs (  
  log_id BIGSERIAL PRIMARY KEY,  
  event_type VARCHAR(100),  
  user_id VARCHAR(50),  
  description TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

PART 2: Central Election Database Schema

Central DB stores only aggregated + reconciled data.

1 centers

sql

```
CREATE TABLE centers (  
  center_id VARCHAR(20) PRIMARY KEY,  
  center_name VARCHAR(150),  
  district VARCHAR(100),  
  total_registered_voters INTEGER  
);
```

2 center_turnout_summary

sql

```
CREATE TABLE center_turnout_summary (  
  center_id VARCHAR(20) REFERENCES centers(center_id),  
  total_printed INTEGER,  
  total_cast INTEGER,  
  synced_at TIMESTAMP,  
  PRIMARY KEY(center_id)  
);
```

3 final_results

sql

```
CREATE TABLE final_results (  
  center_id VARCHAR(20),  
  party_id VARCHAR(20),  
  total_votes INTEGER,  
  verified BOOLEAN DEFAULT FALSE,  
  PRIMARY KEY(center_id, party_id)  
);
```

4 discrepancy_reports

sql

```
CREATE TABLE discrepancy_reports (  
  report_id SERIAL PRIMARY KEY,  
  center_id VARCHAR(20),  
  printed_count INTEGER,  
  cast_count INTEGER,  
  manual_sum INTEGER,  
  status VARCHAR(20) CHECK (status IN ('MATCH','MISMATCH')),  
  generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```



Important Constraints (Very Critical)

Add these:

SCSS

UNIQUE (voter_id) in ballots

So one voter cannot get two ballots.

Also:

SCSS

CHECK (total_cast <= total_printed)

via trigger.

Recommended Indexes

sql

```
CREATE INDEX idx_voter_nid ON voters(nid_number);
CREATE INDEX idx_ballot_status ON ballots(status);
CREATE INDEX idx_audit_user ON audit_logs(user_id);
```

Data Flow Mapping

Action	Table Updated
Biometric success	voting_tokens
Ballot print	ballots + turnout_log
Ballot cast	ballots + turnout_log
Manual count	manual_count_entries
Final reconciliation	final_results



Advanced Improvement (Optional)

If তুমি enterprise-level করতে চাও:

- Partition ballots table by center_id
 - Enable row-level security (PostgreSQL RLS)
 - Add immutable audit ledger table
 - Add hash-chain log integrity
-



তোমার জন্য পরবর্তী কাজ

এখন তুমি করতে পারো:

- 1 ER Diagram আঁকা
 - 2 Trigger লিখা (double voting prevent)
 - 3 Reconciliation stored procedure বানানো
 - 4 Dummy data দিয়ে simulation
-

তুমি কি চাইছো এখন:

- ER Diagram (visual structure)
- Trigger & Stored Procedure
- Complete Backend API design
- Threat model document

বললেই next level এ নিয়ে যাব।

Qr encryption logic example..

ভালো — এখন আমি তোমাকে **Production-style QR Encryption Logic** দেখাচ্ছি।

Goal:

- QR clone করা যাবে না
- Data tamper করলে detect হবে
- Offline verify করা যাবে
- Replay attack prevent হবে

আমি ৪ অংশে দিচ্ছি:

- 1 QR Data Structure
- 2 Encryption + Signing Logic
- 3 QR Generate Example (Java)
- 4 QR Validate Logic

1 QR Data Structure (Plain Payload Before Encryption)

QR-এ সরাসরি data রাখা যাবে না।

Plain JSON (Internal Only):

json

```
{
  "ballot_id": "uuid",
  "center_id": "C102",
  "serial": 458721,
  "timestamp": 1739458200,
  "random_nonce": "a8F92kLm",
  "version": "1.0"
}
```

কেন এগুলো লাগবে?

Field	Purpose
ballot_id	Unique ballot tracking
center_id	Cross-center misuse prevent
serial	Print sequence
timestamp	Expiry check
random_nonce	Prevent predictable pattern
version	Future upgrade support

2 Secure QR Encryption Model

⚠ শুধু AES করলে হবে না

⚠ শুধু hash করলেও হবে না

তাই আমরা করবো:

vbnet

STEP 1 → JSON তৈরি

STEP 2 → AES-256 Encrypt

STEP 3 → HMAC-SHA256 Sign

STEP 4 → Combine → Base64 Encode

STEP 5 → QR Generate

Final QR Payload format:

SCSS

```
Base64(  
    AES_ENCRYPTED_DATA  
    +  
    HMAC_SIGNATURE  
)
```

3 Java Encryption Example (Production Style)

AES + HMAC Example

java

```
import javax.crypto.Cipher;  
import javax.crypto.Mac;  
import javax.crypto.spec.SecretKeySpec;  
import java.util.Base64;  
  
public class QREncryption {  
  
    private static final String AES_KEY = "12345678901234561234567890123456"; // 32 bytes  
    private static final String HMAC_KEY = "hmac-secret-key-123456";
```

```

public static String encryptAndSign(String data) throws Exception {

    // AES Encrypt
    SecretKeySpec keySpec = new SecretKeySpec(AES_KEY.getBytes(), "AES");
    Cipher cipher = Cipher.getInstance("AES");
    cipher.init(Cipher.ENCRYPT_MODE, keySpec);
    byte[] encrypted = cipher.doFinal(data.getBytes());

    // HMAC Sign
    Mac mac = Mac.getInstance("HmacSHA256");
    SecretKeySpec hmacKey = new SecretKeySpec(HMAC_KEY.getBytes(), "HmacSHA256");
    mac.init(hmacKey);
    byte[] signature = mac.doFinal(encrypted);

    // Combine encrypted + signature
    byte[] finalPayload = new byte[encrypted.length + signature.length];
    System.arraycopy(encrypted, 0, finalPayload, 0, encrypted.length);
    System.arraycopy(signature, 0, finalPayload, encrypted.length, signature.length);

    return Base64.getEncoder().encodeToString(finalPayload);
}
}

```

4 QR Generation (ZXing Library)

```

java

import com.google.zxing.qrcode.QRCodeWriter;
import com.google.zxing.common.BitMatrix;

public void generateQR(String encryptedData) throws Exception {
    QRCodeWriter writer = new QRCodeWriter();
    BitMatrix matrix = writer.encode(encryptedData,
                                     com.google.zxing.BarcodeFormat.QR_CODE,
                                     300, 300);
}

```

5 QR Validation Logic (Ballot Box Side)

Scanner side logic:

markdown

1. Base64 decode
2. Split encrypted + signature
3. Recalculate HMAC
4. Compare signature
5. If match → AES decrypt
6. Validate:
 - *center_id matches*
 - *ballot not already CAST*
 - *timestamp valid*

Pseudo-validation:

java

```
if(!receivedSignature.equals(calculatedSignature)) {  
    rejectBallot();  
}
```

6 Extra Security Improvements (Highly Recommended)

✓ Use AES/GCM instead of plain AES

- Built-in authentication

✓ Add Expiry Time

Ballot must be cast within 30 minutes.

✓ Device-specific salt

Each polling center uses different secret.

✓ Secure key storage

Keys never hardcoded.

Use:

- Android Keystore
 - Hardware Security Module (HSM)
-

7 Anti-Cloning Strategy

Even if QR copied:

- Ballot_id UNIQUE in DB
- status must be PRINTED
- After first CAST → status updated
- Second attempt → rejected

Final Secure Flow

mathematica

Ballot Print



Generate Secure QR



Vote Cast



Scan QR



Verify Signature



Decrypt



Check DB status



Increment turnout

তোমার Next Upgrade

চাও কি আমি দেই:

- AES/GCM production-level secure version?
- PostgreSQL trigger to auto reject duplicate ballot?
- Replay attack prevention design?
- Full secure key management architecture?
- Threat model (Red Team style analysis)?

বললেই আমরা system টাকে enterprise-grade level এ নিয়ে যাব.

ভালো — এখন আমি তোমাকে **Android Prototype Flow (End-to-End)** দিচ্ছি।

এটা হবে **1 Polling Center Simulation App** — modular, testable, scalable structure।



ANDROID PROTOTYPE FLOW

(Hybrid Secure Ballot System – Single Center Demo)



1

Overall App Architecture

Architecture Pattern:

- MVVM
- Room (Local DB)
- Repository Layer
- Single Activity + Multiple Fragments

nginx

MainActivity

- └─ LoginFragment
- └─ VoterVerificationFragment
- └─ BallotPrintFragment
- └─ BallotBoxScanFragment
- └─ TurnoutDashboardFragment
- └─ ManualCountingFragment



2

Complete User Flow (Screen-by-Screen)



1. Login Screen

Roles:

- Presiding Officer

- Counting Officer
- Agent

Flow:

pgsql

Enter ID + Password

↓

Check Role

↓

Navigate to Role Dashboard

Room Table → users

● 2. Voter Verification Screen

UI Elements:

- NID Input Field
- Fingerprint Button (simulate)
- Verify Button
- Status Indicator

Flow:

sql

Enter NID

↓

Fetch voter from DB

↓

Fingerprint verify (simulate match)

↓

If has_voted = false

↓

Generate Voting Token

↓

Navigate to BallotPrint

Update:

- voting_tokens table

- audit_logs



3. Ballot Printing Screen

What Happens:

pgsql

Receive Voting Token



Generate Ballot **UUID**



Encrypt Payload



Generate QR



Display QR (simulate print)



Insert into ballots **table**

Update:

- ballots (status = PRINTED)
- turnout_log.total_printed++



4. Ballot Box Scan Screen

Simulate QR scan (camera)

Flow:

lua

Scan QR



Decrypt



Validate:

- center_id **match**

- ballot **status** == PRINTED



Update ballot **status** = CAST

↓

turnout_log.total_cast++

↓

Set voter.has_voted = **true**

If invalid → Show RED alert.



5. Turnout Dashboard (Live View)

Visible To:

- Presiding Officer
- Counting Officer

Shows:

- Total Registered Voters
- Total Ballots Printed
- Total Ballots Cast
- Remaining Voters



No party vote shown



6. Manual Counting Screen (After Poll Close)

Only Enabled When:

poll_status = CLOSED

UI:

- Party List
- Vote Input Field
- Submit Button

Flow:

markdown

Agent enters party vote

↓

Store in manual_count_entries

After all entries:

System runs reconciliation:

sql

Sum(party votes)

==

total_cast ?

If mismatch → show warning.

3 Room Database Structure (Android)

Entities:

nginx

VoterEntity

BallotEntity

TokenEntity

TurnoutEntity

PartyEntity

ManualCountEntity

UserEntity

AuditLogEntity

4 App Navigation Logic

MainActivity

- Controls session
- Controls poll state (OPEN / CLOSED)

Poll States:

mathematica

OPEN

- Verification allowed
- Printing allowed
- Scan allowed

CLOSED

- Verification disabled
- Manual counting enabled



Security Simulation Layer

For Prototype:

- Hardcoded AES key
- Simulated fingerprint match
- Offline only
- Local DB only

Later Upgrade:

- Android Keystore
- Real biometric API



Complete Prototype Workflow Diagram

mathematica

Login



Voter Verification



Token Generation



Ballot Print (QR)



Vote Cast



QR Scan



Turnout Update



Poll Close



Manual Count Entry



Reconciliation Check



7

Suggested Tech Stack

Feature	Tool
DB	Room
QR	ZXing
Encryption	Java Crypto
Architecture	MVVM
Dependency Injection	Hilt (optional)



8

Simulation Plan (Testing)

Test Cases:

- Double vote attempt
- Invalid QR
- Expired token
- Ballot clone attempt
- Manual count mismatch



9

Project Folder Structure

pgsql

data/

dao/

entity/

repository/

ui/

login/

verification/

ballot/

scan/

dashboard/

counting/

security/

EncryptionUtil

QRUtil

utils/

Realistic Advice For You

Since তুমি Android project করছো (CalculatorEasy, WomenSafety),
এইটা তুমি final year project হিসেবে করতে পারো:

Scope Limit করো:

- 1 Center
- 50 Dummy Voters
- Simulated biometric
- Offline only

চাও কি আমি:

- Complete Android Project Structure code skeleton দেই?
- Room Entity + DAO ready code দেই?
- Navigation graph design করি?
- UI XML structure বানিয়ে দেই?
- Prototype Report format বানিয়ে দেই?

Next কোনটা?

Bagladesh er moto samajik poristhiti te jekhane gram onchole
kendro dokhol + vote riging+ sate e vote Engineering hoy
sekhane ei poddoti bebohar kora kototuk thik hobe?
- kibhabe machine gular nirapotta briddhi kora jay?

তোমার প্রশ্নটা খুব বাস্তবভিত্তিক।

যে সামাজিক প্রেক্ষাপটে গ্রামাঞ্চলে **কেন্দ্র দখল, জাল ভোট, ভয়ভীতি, সংঘবদ্ধ প্রভাব** থাকে — সেখানে কেবল প্রযুক্তি দিয়ে সব সমস্যা সমাধান হয় না।

আমি বিষয়টা দুই ভাগে বিশ্লেষণ করছি:

☐ এই পদ্ধতি কতটা কার্যকর হতে পারে?

✓ যেখানে এটা সাহায্য করবে

1. জাল ভোট কমানো

- Biometric verification থাকলে “একজনের নামে আরেকজন ভোট” কঠিন হবে
- ১ ভোটার = ১ ballot enforced

2. Ballot stuffing কমানো

- QR validated ballot ছাড়া turnout বাড়বে না
- Duplicate ballot reject হবে

3. Immediate turnout tracking

- কতজন ভোট দিয়েছে — রিয়েল-টাইম জানা যাবে
- পরে সংখ্যা প্লটানো কঠিন হবে

✗ কিন্তু যেসব সমস্যা প্রযুক্তি দিয়ে থামানো যায় না

1. কেন্দ্র দখল (Booth Capture)

যদি সশস্ত্র দল এসে পুরো কেন্দ্র নিয়ন্ত্রণ নেয়:

- তারা মেশিন অপারেটরকেই বাধ্য করতে পারে
- ভোটারদের ঢুকতে না দিতে পারে
- দলীয়ভাবে ভোট দিয়ে দিতে পারে

👉 এখানে প্রযুক্তি না, নিরাপত্তা বাহিনী ও প্রশাসনিক নিয়ন্ত্রণ দরকার।

2. ভোটের ভয়ভীতি

Machine secure হলেও:

- ভোটের ভয় পেলে ভোট দিতে নাও আসতে পারে
 - জোর করে ভোট করানো সম্ভব
-

3. ভিতরের লোকের দুর্নীতি

Presiding officer বা staff যদি দলীয় প্রভাবিত হয়:

- সিস্টেম অপব্যবহার হতে পারে
 - Poll close আগে manual override
-

উপসংহার

এই পদ্ধতি:

- জাল ভোট কমাবে
- ভোটা বিকৃতি কমাবে
- পরবর্তী অডিট সহজ করবে

কিন্তু:

- রাজনৈতিক সহিংসতা থামাবে না
 - প্রশাসনিক দুর্বলতা ঠিক করবে না
-

☐☐ মেশিন নিরাপত্তা কিভাবে বাড়ানো যায়?

এখানে আমি ৩ স্তরের নিরাপত্তা দিচ্ছি:

A. Physical Security Layer

✓ Tamper-Proof Casing

- Screw seal
- Open করলে auto tamper log

✓ Unique Physical Seal Number

- প্রতিটি মেশিনের seal number

- Poll start এ record করা
- Poll close এ মিলিয়ে দেখা

✓ CCTV Coverage

- Ballot print area
 - Ballot box insertion area
-

B. Software Security Layer

✓ Secure Boot

Device firmware পরিবর্তন করা যাবে না

✓ Encrypted Storage

- Full disk encryption
- Ballot DB encrypted

✓ Write-Once Audit Log

- Log delete করা যাবে না
- Hash-chain system

✓ Offline First Design

- Internet ছাড়াও কাজ করবে
 - পরে sync
-

C. Cryptographic Security Layer

✓ Center-Specific Key

প্রতি কেন্দ্রের আলাদা encryption key

✓ Ballot Expiry Window

Printed ballot 30 মিনিটের মধ্যে cast না হলে invalid

✓ Replay Detection

ballot_id UNIQUE constraint

✓ Digital Signature

D. Operational Safeguards (সবচেয়ে গুরুত্বপূর্ণ)

এগুলো ছাড়া প্রযুক্তি কাজ করবে না:

1. Multi-party উপস্থিতি

সব দলের agent উপস্থিত না থাকলে মেশিন চালু হবে না

2. Live Public Display

Turnout public screen এ visible

3. Post-election Audit

Random center audit

4. Parallel Manual Record

ডিজিটাল + কাগজ — দুইটা থাকবে

☐☐ বাস্তবতা মূল্যায়ন (Bangladesh Context)

বাংলাদেশের মতো পরিবেশে:

Risk	Tech Effectiveness
Fake vote	High prevention
Booth capture	Low prevention
Staff bias	Medium prevention
Result tampering	High prevention
Violence	No prevention

☐☐ সবচেয়ে নিরাপদ বাস্তব মডেল

আমি যদি বাস্তবসম্মত সমাধান দেই:

✓ Full electronic না

✓ Full manual না

👉 Hybrid system:

- Biometric verification
- Paper ballot mandatory
- QR turnout tracking
- Manual counting
- Strong armed security presence

📄 তোমার জন্য গুরুত্বপূর্ণ চিন্তা

প্রযুক্তি একা নির্বাচন সং করতে পারে না।

প্রযুক্তি কেবল:

- প্রমাণ রেখে দেয়
- জালিয়াতি detect সহজ করে
- পরবর্তী আইনি পদক্ষেপ সহজ করে

তুমি চাইলে আমি এখন বিশ্লেষণ করতে পারি:

- Worst-case attack scenario (Red Team Analysis)
- Booth capture হলে system কীভাবে react করবে?
- Insider attack model
- অথবা Military-grade security architecture design

কোন দিকে যেতে চাও?

Worst-case attack scenario (Red Team Analysis)
Koro

ভালো — এখন আমরা তোমার Hybrid Secure Voting System-কে **Red Team perspective** থেকে ভাঙার চেষ্টা করবো।

ধরো আক্রমণকারী খুব সংগঠিত, রাজনৈতিকভাবে প্রভাবশালী, এবং ভিতরে/বাইরে দুই জায়গাতেই অ্যাক্সেস আছে।

আমি বিশ্লেষণ করছি ৬টি Worst-Case Scenario দিয়ে:

SCENARIO 1: Full Booth Capture + Forced Voting

Attack Flow

1. সশস্ত্র দল কেন্দ্র দখল করে
2. প্রকৃত ভোটারদের ঢুকতে দেয় না
3. Presiding officer কে বাধ্য করে
4. Dummy voter ID ব্যবহার করে biometric bypass (inside help থাকলে)
5. Rapid ballot printing + casting

Impact

- Turnout দ্রুত বেড়ে যায়
- সব ballot valid QR সহ cast হয়
- System বুঝতে পারে না coercion হয়েছে

Why Tech Fails Here

- Biometric আছে, কিন্তু যদি staff জোর করে ভোট করায়
- Physical force cryptography ভাঙে না — কিন্তু democracy ভাঙে

Mitigation

- Panic lockdown mode (hidden button)
 - Sudden abnormal voting rate detection
 - Military-grade external security presence
-

SCENARIO 2: Insider Key Compromise

Attack Flow

1. Center encryption key leak হয়
2. QR structure reverse engineer করা হয়
3. Fake ballot QR generate করা হয়
4. Cast করার সময় system valid ধরে

Impact

- Fake ballot injection
- Digital audit bypass

Root Cause

- Hardcoded key
- Weak key storage

Mitigation

- Hardware Security Module (HSM)
 - Per-ballot ephemeral key
 - Secure element chip
-

SCENARIO 3: Pre-Poll Malware Injection

Attack Flow

1. Poll এর আগেই firmware modify
2. System silently:
 - Certain party votes reject
 - Random invalidation
 - Hidden counter manipulation

Impact

- Manual count mismatch
- Result distortion

Why Dangerous?

- Visible error নেই
- Long-term stealth attack

Mitigation

- Secure boot chain
 - Firmware hash public verification
 - Independent code audit
-

SCENARIO 4: Coordinated Replay Attack

Attack Flow

1. Valid QR scan করে clone করা
2. Before status update, multiple fast scan attempt
3. System race condition exploit

Impact

- Duplicate vote counted

Mitigation

- Atomic DB transaction
 - Ballot status lock
 - Millisecond-level duplicate check
-

SCENARIO 5: Central Server Compromise

Attack Flow

1. Central database hack
2. Aggregated turnout alter
3. Discrepancy reports delete

Impact

- National-level manipulation

Mitigation

- Offline center independence
 - Public cryptographic result publication
 - Multi-location immutable log
-

SCENARIO 6: Psychological Engineering

এটাই সবচেয়ে ভয়ংকর।

Attack Flow

1. Rumor ছড়ানো — “machine hacked”
2. Social media disinformation
3. Public trust collapse

Impact

- Legitimate result questioned
- Political instability

Reality:

Even perfectly secure system → trust crisis হলে collapse করে।

Most Dangerous Combined Attack

Worst-case hybrid attack:

markdown

Booth capture

+

Insider key leak

+

Central log manipulation

+

Disinformation campaign

এই combination হলে system technically secure হলেও legitimacy হারাবে।

Risk Severity Matrix

Attack Type	Likelihood	Damage
Booth Capture	High	Extreme
Insider Threat	Medium	Extreme
Malware Injection	Low-Medium	Severe
QR Clone	Medium	Moderate
Central Hack	Medium	Severe
Disinformation	Very High	Extreme

Hard Truth

Technology prevents:

- Duplicate ballot

- Digital tampering
- Data alteration

Technology cannot prevent:

- Armed coercion
 - Political control
 - Administrative bias
 - Fear-based voter suppression
-



If You Want Military-Grade Design

Then you need:

1. Air-gapped machines
 2. Public cryptographic verification (open audit)
 3. Independent third-party audit
 4. Randomized polling audit
 5. Tamper-evident hardware
 6. Distributed trust (no single admin key)
-



Strategic Conclusion (Bangladesh Context)

Hybrid system + strong security presence ছাড়া

purely tech-based solution যথেষ্ট না।

Systemকে design করতে হবে এই mindset নিয়ে:

“Assume staff compromised. Assume attacker organized. Assume internet hostile.”

তুমি চাইলে এখন আমরা করতে পারি:

- Insider attack deep analysis
- Booth capture technical counter-design
- Zero-trust election architecture
- অথবা full national-level resilient system design

কোন level এ যেতে চাও?

Booth capture হলে system কীভাবে react করবে?+ booth capture hoise kibhabe bujbe. Cz bd te jonosonkha onek. Onek maush vote dite jay. Detect koara upay ki?

খুব গুরুত্বপূর্ণ প্রশ্ন।

আমি সরাসরি বলছি:

👉 **Booth capture** পুরোপুরি প্রযুক্তি দিয়ে থামানো যায় না

কিন্তু **detect + slow down + evidence preserve** করা যায়।

আমি ৩ ভাগে ব্যাখ্যা করছি:

📋 Booth Capture হলে System কীভাবে বুঝবে?

তুমি ঠিক বলছো — বাংলাদেশে জনসংখ্যা বেশি, ভোটের অনেক। তাই “অনেক মানুষ আসছে” আর “জোর করে ভোট করানো হচ্ছে” — এই দুইটা আলাদা করা কঠিন।

তাই আমাদের detect করতে হবে **behavior pattern** দিয়ে।

🧠 A. Abnormal Voting Rate Detection

Normal pattern (গ্রামাঞ্চল)

- সকাল ৮–১০ → বেশি ভিউ
- দুপুরে কম
- বিকালে আবার বাড়ে

Suspicious pattern

- হঠাৎ ৫ মিনিটে ৫০–১০০ ভোট
- ধারাবাহিক high-speed voting (no gap)
- একই operator দ্রুত verify করছে

Detection Logic

markdown

```
if (votes_per_minute > threshold)
    flag = HIGH_RISK
```

Example:

- Average = 1 vote / 40–60 sec
- Suddenly = 1 vote / 5 sec

→ abnormal



B. Biometric Pattern Anomaly

Booth capture হলে সাধারণত:

- একই staff verify করছে
- Fingerprint mismatch rate হঠাৎ বেড়ে যায়
- Face scan bypass mode activate

Detect:

```
SCSS
```

```
if (fingerprint_fail_rate > normal_threshold)
    alert
```



C. Continuous Non-stop Voting Without Natural Break

মানুষ:

- কথা বলে
- ভুল করে
- সময় নেয়

Booth capture হলে:

- Mechanical rhythm
- Almost constant interval

AI-based pattern:

```
sql
```

Calculate **time interval** variance

Low variance = suspicious

☐☐ Booth Capture হলে System কীভাবে React করবে?

এখন detection হয়েছে — এবার reaction।

● Level 1: Silent Alert Mode

- Screen এ visible না
 - Central server এ alert পাঠাবে
 - “Abnormal Voting Rate Detected”
-

● Level 2: Slowdown Mode

System automatically:

- Biometric verification delay করবে (random delay 10–20 sec)
- Extra confirmation চাইবে
- Random captcha-like human check

→ Forced rapid voting slow হয়ে যাবে

● Level 3: Panic Lock Mode

Hidden combination button:

- Machine lock
- Ballot print stop
- Emergency code required

Used only if staff under threat.

● Level 4: Automatic Audit Trigger

If:

High-speed voting for 10 mins+

Then:

- Mark center as “High Risk”
- Mandatory full manual recount
- Independent investigation

🇬🇧 Bangladesh Context – Practical Detection Strategy

কারণ জনসংখ্যা বেশি, তাই শুধু speed দিয়ে detect করা যাবে না।

তাই Multi-signal detection দরকার:

Signal	Why Important
Voting rate spike	Rapid stuffing
Biometric failure spike	Forced override
Operator-only activity	Staff-only voting
Zero natural break	Mechanical pattern
Same device ID continuous	No voter variability

🔍 Smart Detection Model (Simple Math Version)

markdown

Suspicion Score =
(Speed weight × abnormal speed)
+ (Failure rate weight × biometric anomaly)
+ (Pattern weight × time variance anomaly)

If score > threshold → High Risk

Reality Check

Booth capture হলে:

- Armed presence থাকলে
- Staff ভয় পেলে
- Voters ঢুকতে না পারলে

Machine detect করতে পারে,

কিন্তু stop করতে পারবে না — unless:

- External law enforcement act করে

☐ Stronger Defensive Design

✓ Separate Verification & Ballot Print Operators

একজন সব না করুক।

✓ Mandatory Multi-party Presence Sensor

Agent login ছাড়া print disable

✓ Randomized Voting Delay

System always exact timing না রাখে

✓ Public Turnout Display Outside Center

হঠাৎ spike হলে সবাই দেখবে

সবচেয়ে কার্যকর বাস্তব সমাধান

Hybrid Control Model:

- Tech detection
- Armed neutral security presence
- Live public turnout transparency
- Mandatory audit of flagged centers

কঠিন সত্য

Technology can:

- Detect anomalies
- Create evidence
- Slow attackers

Technology cannot:

- Fight guns
- Replace political neutrality
- Replace state authority

তুমি চাইলে এখন আমরা করতে পারি:

- Mathematical anomaly detection model design
- AI-based pattern detection system
- অথবা “Zero Trust Election Architecture” design

কোনটা explore করতে চাও?

Insider attack model

অথবা Military-grade security architecture design

তুমি যেটা জানতে চাইছো — এটা election system design-এর সবচেয়ে কঠিন অংশ।

কারণ ইতিহাস দেখায়: বেশিরভাগ বড় সিস্টেম ভাঙে insider দিয়ে, outsider দিয়ে না।

আমি দুইটা জিনিস দিচ্ছি:

1 Insider Attack Model (Red Team View)

2 Military-Grade Security Architecture (Counter Design)

PART 1: Insider Attack Model (Worst Case)

Assume:

- Presiding officer compromised
 - IT admin compromised
 - অথবা local technician bribed
-

Insider Attack Type 1: Silent Ballot Inflation

Attack Flow

1. Staff late evening এ dummy voter verify করে
2. Ballot print করে
3. QR scan করে cast করে
4. Real voter পরে আসলে — already voted দেখায়

Why এটা dangerous?

- সব QR valid
- সব DB consistent
- Manual count মিলেও যাবে

👉 এটা detect কঠিন যদি coercion না থাকে।

Insider Attack Type 2: Selective Rejection

1. নির্দিষ্ট এলাকার voter biometric “fail” দেখানো
2. Delay করে নিরুৎসাহ করা
3. অন্যদের দ্রুত allow করা

Subtle suppression।

Insider Attack Type 3: Key Leakage

1. Center encryption key leak
 2. QR structure বাইরে যায়
 3. Fake ballot তৈরি সম্ভব
-

Insider Attack Type 4: Log Manipulation

1. Audit log delete
 2. Timestamp alter
 3. Manual count entry modify
-

Insider Attack Type 5: Firmware Backdoor

1. Poll আগে malicious firmware

2. নির্দিষ্ট pattern detect করে internal counter adjust

Insider Risk Matrix

Attack	Detectability	Damage
Dummy Voting	Low	Extreme
Selective Rejection	Very Low	High
Key Leak	Medium	Extreme
Log Delete	Medium	High
Firmware Backdoor	Very Low	Catastrophic

PART 2: Military-Grade Security Architecture

এখন counter-design |

Principle:

Assume insider compromised.

1 Zero-Trust Election Architecture

Core Idea:

- No single human trusted
 - No single device trusted
 - No single key trusted
-

A. Split Authority Model

Instead of:

One officer → full control

Use:

Function	Controlled By
Verification	Officer A
Ballot Print	Officer B
Ballot Scan	Officer C

No single person can complete full cycle.

B. Multi-Key Cryptography (Shamir Secret Sharing)

Encryption key split into 3 parts:

- Election Commission key
- Judiciary key
- Opposition observer key

All 3 needed to reconstruct master key.

C. Hardware Root of Trust

Use:

- TPM chip
- Secure Enclave
- Hardware-backed key storage

Firmware must:

- Boot only if signature valid
 - Hash publicly verifiable
-

D. Immutable Audit Ledger

Use:

Hash chaining:

mathematica

$\text{Hash}(n) = \text{SHA256}(\text{previous_hash} + \text{event_data})$

Log modify করলে পুরো chain break হবে।

E. Air-Gapped Core Logic

- Ballot counting device → no internet
- Sync only via signed USB
- USB auto verify digital signature

F. Real-Time Public Cryptographic Proof

After poll close:

System publishes:

mathematica

Center Hash Digest

Anyone can verify no alteration হয়েছে।

Advanced Insider Detection Mechanism

1 Operator Behavior Profiling

Track:

- Speed variance
- Error rate
- Time-of-day anomaly

If deviation > threshold → flag

2 Random Mandatory Re-authentication

Every X ballots:

- Staff re-authenticate biometric

- Multi-party approval required
-

Automatic Cross-Center Statistical Audit

If:

- One center turnout 95%
- Nearby centers 60%

→ automatic investigation

Most Secure Model (Realistic Hybrid)

1. Biometric verification
 2. Paper ballot mandatory
 3. Encrypted QR turnout
 4. Manual counting
 5. Random center forensic audit
 6. Independent third-party firmware audit
-

Hard Truth

Military-grade architecture:

- Expensive
- Complex
- Requires institutional neutrality
- Needs public trust

Technology alone = insufficient

Institutional integrity = mandatory

Strategic Advice (Bangladesh-like Context)

If I were designing realistically:

✓ Focus on:

- Detectability

- Public transparency
- Tamper evidence

✗ Not on:

- Fully automated result
-

তুমি চাইলে এখন আমি করতে পারি:

- Full Zero-Trust Election System diagram
- Cryptographic public verification design
- Insider threat simulation with math model
- অথবা National-scale resilient election blueprint

কোন দিকে deep dive করবো?